

Plasticity Is All You Need?

A Testable Proposal for Persistent Fast-Weight Adaptation in Neural Architectures

Thomas Nauheimer

Cognitive Systems Research • Nierstein, Germany • nauheimer.t@gmail.com

Abstract

We investigate a frozen linear transformation augmented with persistent low-rank fast weights. Unlike external sidecar or retrieval-augmented architectures, the supplied PlasticLinearProjected adapter participates directly in a PyTorch model's forward computation. It applies explicit target-driven delta-rule updates and protects a specified input subspace $\text{span}(Q)$ via orthogonal projection. Thin QR factorizations and a small-core SVD compress updates without explicitly reconstructing the dense fast-weight matrix. Synthetic linear-regression experiments demonstrate online adaptation, preservation of protected inputs within floating-point precision ($\leq 10^{-16}$), unchanged base weights, and deterministic state serialization. These experiments establish implementation correctness in a testable, reproducible framework for continual fast-weight adaptation.

1. Relationship to Existing Work

Fast weights, test-time adaptation, and trainable plasticity have established strong theoretical precedents in modern machine learning:

- **Fast Weights & Linear Attention:** Ba et al. (2016) explored fast-weight memory for recurrent networks; Schlag et al. (2021) demonstrated that linearized self-attention mechanisms operate as fast-weight programmers.
- **Subspace & Gradient Projection:** Orthogonal Weights Modification (OWM; Zeng et al., 2019) and Gradient Projection Memory (GPM; Saha et al., 2021) project gradient updates onto the orthogonal complement of protected feature subspaces to mitigate catastrophic forgetting.
- **Parameter vs. Activation Orthogonality (O-LoRA):** Wang et al. (2023; O-LoRA) enforce parameter-space orthogonality across discrete sequential tasks. In contrast, our method projects directly in activation space (onto the nullspace of historical activations) and operates continuously without task boundaries.
- **Test-Time Training (TTT):** Recent architectures such as TTT-Linear (Sun et al., 2024) and Titans (Behrouz et al., 2024) optimize hidden states via inference-time inner-loop updates.
- **Differentiable Plasticity & EWC:** Miconi et al. (2018) optimized plastic connections via outer-loop meta-learning, while Kirkpatrick et al. (2017) utilized Fisher information matrices to penalize changes to important parameters.

2. Implemented Model and Mathematical Formulation

For row-batched inputs $X \in \mathbb{R}^{N \times d_{\text{in}}}$, the forward mapping is defined as:

$$A(t) = U(t) V(t)^T$$

$$Y = X W_{\text{slow}}^T + b + \gamma (X V) U^T$$

where W_{slow} is the frozen base parameter matrix, $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{d_{\text{in}} \times r}$ are dynamic low-rank factor buffers initialized to zero, and $\gamma \in [0, 1]$ scales adaptation energy. Given target activations T , error residual $E = T - Y$, and orthogonal subspace projector $P = I - Q Q^T$, the candidate update satisfies:

$$\mathbf{A}_{\text{cand}} = (\mathbf{1} - \lambda) \mathbf{A} + (\eta \gamma / \mathbf{N}) \mathbf{E}^T \mathbf{X} \mathbf{P}$$

In deep language models, deriving verified intermediate target activations T remains an unsolved integration requirement; unverified model generations must not be treated automatically as correct learning targets.

3. Subspace Protection & Invariance Properties

Let $Q \in \mathbb{R}^{d_{\text{in}} \times k}$ denote an orthonormal basis for protected input features. If $A_0 Q = 0$ and every successive update is right-projected by $P = I - Q Q^T$, then $A_t Q = 0$ holds identically in exact arithmetic. Consequently, the adapter leaves the linear layer output strictly invariant for all inputs in $\text{span}(Q)$.

Critical Qualification: Subspace invariance is an algebraic property of the projector P by construction; it protects only inputs residing strictly in $\text{span}(Q)$. In deep autoregressive models, natural token activations frequently exhibit orthogonal components which undergo adaptation. Furthermore, every dimension allocated to Q reduces available rank capacity for novel associations.

4. Low-Rank Compression via Thin QR & Core SVD

Representing candidate rank updates as $L R^T$ with $m = r + N$ columns:

- $L = [\sqrt{(1-\lambda)} U, \sqrt{(\eta \gamma / N)} E^T] \in \mathbb{R}^{d_{\text{out}} \times m}$
- $R = [\sqrt{(1-\lambda)} P V, \sqrt{(\eta \gamma / N)} P X^T] \in \mathbb{R}^{d_{\text{in}} \times m}$

Thin QR factorizations $L = Q_L R_L$ and $R = Q_R R_R$ reduce the SVD to the small core matrix $R_L R_R^T \in \mathbb{R}^{m \times m}$. Total adaptation step complexity is $\mathcal{O}((d_{\text{in}} + d_{\text{out}}) m^2 + m^3)$. Crucially, the dense dimension product $d_{\text{in}} \times d_{\text{out}}$ never appears in the computational complexity, as full weight matrices are never materialized. Forward overhead remains $\mathcal{O}(N r (d_{\text{in}} + d_{\text{out}}))$.

5. Empirical Validation & Reproducibility

We evaluated *PlasticLinearProjected* across five independent random seeds (CPU, float64, 32-to-16 projection, rank 8, 4 protected directions, 8 novel adaptation directions, 160 updates). The exact results below are reproducible via python examples/reproduce_benchmark.py (verified with tests/test_projected_delta.py):

Seed	Held-out MSE Before	Held-out MSE After	Max Protected Change	Invariance Check
0	0.254999	2.74e-10	3.47e-17	Exact (AQ=0)
1	0.291153	3.13e-10	2.78e-17	Exact (AQ=0)
2	0.227038	2.44e-10	2.78e-17	Exact (AQ=0)
3	0.413457	4.44e-10	9.02e-17	Exact (AQ=0)
4	0.501408	5.38e-10	3.47e-17	Exact (AQ=0)

6. Production Integration Boundaries & Conclusion

Integrating fast weights into production language models requires: (1) direct tensor contraction access rather than external API wrappers, (2) KV-cache coherence management during autoregressive decoding, and (3) systematic derivation of intermediate layer target activations T . The complete PyTorch implementation and unit test suite are released under the MIT License at: <https://github.com/nauheimervater/plastic-transformer>.